

INTRODUCTION

1.1 Introduction

Weather monitoring and forecasting play a crucial role in various sectors such as agriculture, disaster management, transportation, aviation, environmental monitoring, and smart city development. Accurate and timely weather information helps in making informed decisions and reducing the impact of adverse weather conditions.

Conventional weather forecasting systems generally rely on large-scale meteorological stations and numerical weather prediction models, which often fail to provide accurate hyperlocal forecasts due to their limited spatial resolution and high computational requirements. Furthermore, many rural and remote areas lack access to real-time localized weather information.

Recent advancements in Internet of Things (IoT), Artificial Intelligence (AI), and Machine Learning (ML) have enabled the development of intelligent weather monitoring systems capable of collecting real-time environmental data and generating short-term forecasts with improved efficiency and lower implementation costs.

The proposed Intelligent Weather Station and Predictive AI Portal System is an integrated IoT and AI-based weather forecasting platform designed for hyperlocal weather monitoring and short-term prediction. The system utilizes DHT11 sensors to measure temperature and humidity and a DIY anemometer sensor to measure wind speed. The collected sensor data is transmitted to a FastAPI-based backend server through WebSocket communication and stored in an SQLite database for further analysis.

Machine Learning models based on XGBoost regression algorithms are employed to predict temperature and humidity conditions fifteen minutes in advance. The system also incorporates an emergency advisory broadcasting mechanism, live weather visualization using animated particle rendering, and real-time dashboard interfaces developed using Next.js and React.

The developed system provides real-time weather monitoring, predictive analytics, emergency weather communication, and an interactive user interface, making it suitable for hyperlocal weather forecasting applications, smart agriculture, educational research, environmental monitoring, and disaster management systems.

1.2 Need for the Project

- To provide low-cost hyperlocal weather monitoring and forecasting.
- To generate real-time localized weather predictions using AI and Machine Learning.
- To overcome the limitations of traditional large-scale weather forecasting systems.
- To provide early weather information and advisory notifications.
- To assist agriculture, disaster management, and environmental monitoring applications.

- To develop a scalable and intelligent weather station using IoT technologies.

1.2 Problem Statement

Traditional weather forecasting systems often fail to provide accurate short-term forecasts for specific local regions due to insufficient localized observations, limited accessibility, and high implementation costs. There is a need for an intelligent, low-cost, and scalable weather monitoring system capable of collecting real-time environmental data and generating reliable hyperlocal weather forecasts using Artificial Intelligence and Machine Learning techniques.

1.4 Aim of the Project

To design and develop an Intelligent Weather Station and Predictive AI Portal System capable of real-time weather monitoring and generating fifteen-minute advance hyperlocal weather forecasts using IoT sensors, WebSocket communication, and Machine Learning algorithms.

1.5 Objectives of the Project

- To collect real-time weather data using DHT11 and anemometer sensors.
- To establish real-time communication using WebSocket technology.
- To develop Machine Learning models for temperature and humidity prediction.
- To generate fifteen-minute advance weather forecasts.
- To provide emergency advisory broadcasting and weather alert mechanisms.
- To create a low-cost, scalable, and intelligent hyperlocal weather monitoring system.

LITERATURE REVIEW

Weather forecasting has evolved significantly with the advancement of Artificial Intelligence (AI), Internet of Things (IoT), and Machine Learning (ML) technologies. Traditional weather prediction systems primarily rely on numerical weather prediction models and large-scale meteorological stations, which often lack the capability to provide accurate hyperlocal forecasts. Recent research has focused on integrating sensor networks, real-time communication technologies, and machine learning algorithms to improve forecasting accuracy and accessibility.

2.1 Review of Paper

Paper 1: Evolution from Traditional Forecasting to Machine Learning Approaches

Early studies relied heavily on statistical methods such as Auto-Regressive (AR), Moving Average (MA), and ARIMA models for weather prediction. The study by Gebretinsae et al. (2022) demonstrated that machine learning-based forecasting methods could outperform traditional NWP models in terms of computational efficiency and deployment cost. Their work showed that ARIMA and ANN models can effectively predict short-term weather conditions using historical meteorological data.

However, statistical models assume linear relationships among variables and struggle to capture the highly nonlinear behavior of atmospheric conditions. Consequently, researchers began exploring Artificial Neural Networks (ANNs) and deep learning approaches that can model complex weather patterns more effectively.

This transition from statistical forecasting to machine learning represents a significant advancement, but it also introduces challenges related to training complexity, data requirements, and model interpretability.

Paper 2: Integration of IoT and Wireless Sensor Networks

A major trend in recent research is the use of IoT and Wireless Sensor Networks for collecting real-time meteorological data. Studies by Singh et al. (2024) and Bhavani et al. (2025) proposed systems that deploy distributed sensors to measure environmental parameters such as temperature, humidity, pressure, rainfall, and wind speed.

Both studies concluded that IoT-based systems provide significant advantages over traditional weather stations because they:

- Enable continuous monitoring.
- Support localized weather prediction.
- Reduce infrastructure costs.
- Improve accessibility of weather information.

However, the studies differ in their implementation approaches. Singh et al. focused on WSN architecture and machine learning integration, whereas Bhavani et al. emphasized cloud computing and user-friendly dashboards. Despite these advancements, both studies primarily addressed data collection and visualization rather than improving predictive intelligence.

Therefore, while IoT technologies have solved the problem of data acquisition, the challenge of generating highly accurate and localized forecasts remains an open research problem.

Paper 3: Comparison of Machine Learning and Deep Learning Models

Several machine learning techniques have been explored for weather prediction, including ANN, Random Forest, LSTM, and BiLSTM models [4, 5].

The work by Kukre et al. (2025) employed Random Forest Regressor and Random Forest Classifier models for temperature and rainfall prediction. The authors reported satisfactory prediction accuracy while maintaining relatively low computational complexity. Random Forest models are particularly advantageous because they require less training time and are easier to deploy on low-resource systems.

Conversely, Isler et al. (2025) demonstrated that deep learning models significantly outperform conventional machine learning algorithms. Their study compared W-ANN, W-LSTM, and W-BiLSTM models and found that the Wavelet-BiLSTM model achieved approximately 97% prediction accuracy.

The comparison between these studies reveals an important trade-off:

- Random Forest models are computationally efficient and suitable for embedded devices.[4]
- BiLSTM models provide superior prediction accuracy but require significantly greater computational resources and larger datasets.

These findings indicate that no single model completely satisfies the requirements of accuracy, computational efficiency, and real-time deployment simultaneously.

Paper 4: Fog Computing and Edge Intelligence

Another emerging research direction is the integration of fog computing with IoT-based weather forecasting systems. Isler et al. introduced a fog-enabled architecture where weather data is processed closer to the sensor nodes rather than entirely in the cloud.

Compared with traditional cloud computing approaches, fog computing offers:

- Reduced latency.
- Lower bandwidth consumption.
- Faster response time.
- Improved real-time processing. [5]

However, fog computing systems introduce additional challenges such as resource allocation, edge security, and model optimization for low-power devices.

Most existing studies still rely heavily on cloud infrastructure and have not sufficiently explored lightweight edge-based forecasting methods.

Paper 5: Weather Forecasting for Agriculture and Smart Farming

Agriculture is among the sectors most vulnerable to weather variations [6]. Das and Nayak (2025) proposed a conceptual framework integrating IoT, AI, and machine learning for localized agricultural weather forecasting.

Their study emphasized:

- Real-time meteorological monitoring.
- Crop-specific decision support.
- Resource optimization.
- Climate-resilient agriculture.

The authors argued that existing weather forecasting systems often fail to provide localized and crop-specific recommendations, particularly in rural areas.

Their findings complement previous IoT-based studies by highlighting that accurate prediction alone is insufficient; decision-support mechanisms are equally important for practical applications.

2.2 Comparison of Existing ML Algorithms

Algorithm	Advantages	Limitations
ARIMA	Good for time-series forecasting	Poor with nonlinear patterns
ANN	Learns complex relationships	Requires large datasets
LSTM	Captures temporal dependencies	Computationally expensive
BiLSTM	Higher accuracy than LSTM	More training time
Random Forest	Fast, robust, interpretable	Limited sequence learning
Hybrid Wavelet + BiLSTM	Highest accuracy	Higher implementation complexity

2.3 Research Gaps

- Existing systems primarily focus on large-scale forecasting and provide limited hyperlocal predictions.
- Many weather monitoring systems do not incorporate real-time machine learning-based forecasting.
- Most low-cost weather stations lack intelligent predictive capabilities and emergency alert mechanisms.
- Limited studies integrate IoT sensors, WebSocket communication, machine learning prediction, and real-time visualization into a single platform.
- Existing systems often require expensive infrastructure and high computational resources.

2.4 Feasibility Analysis

Technical Feasibility

- ESP32/NodeMCU available at low cost.
- Sensors (DHT22, BMP280, Rain Sensor, Wind Sensor) are readily available.
- Python, TensorFlow, Scikit-Learn support ML implementation.
- Cloud platforms such as ThingsBoard and ThingSpeak are free or low-cost.

Economic Feasibility

- Estimated hardware cost: ₹2500–₹5000.
- Open-source software tools reduce development cost.

Time Feasibility

- Literature Survey: 2 weeks
- Hardware Setup: 3 weeks
- Data Collection: 4–6 weeks
- ML Model Development: 3 weeks
- Testing and Documentation: 2 weeks

Operational Feasibility

- Useful for agriculture, disaster management, and smart cities.
- Easy deployment in rural and urban areas.

REQUIREMENT

3.1 Hardware Requirements

- Wi-Fi or Ethernet Connectivity
- DHT11 Temperature and Humidity Sensor
- DIY Cup Anemometer Wind Speed Sensor*

*To build a DIY anemometer we require the following components

Component	Quantity	Purpose
Arduino UNO	1	Data acquisition
DC Generator	1	Produces voltage proportional to wind speed
10 k Ω Resistor	1	Pull-down resistor
5.1 V Zener Diode	1	Overvoltage protection
100 μ F Capacitor	1	Smooth voltage fluctuations
Breadboard	1	Prototyping
Jumper Wires	Several	Connections
USB cable	1	Arduino Programming

3.2 Software Requirements

Frontend Technologies: Next.js 15+, React 19, HTML5, CSS3, JavaScript

Backend Technologies: FastAPI, Uvicorn, WebSocket

Database: SQLite

Machine Learning Libraries: Pandas, NumPy, Scikit-learn, XGBoost, Joblib, Matplotlib

IDE: Antigravity, Arduino IDE

METHODOLOGY

4.1 Overview

The methodology adopted for the Intelligent Weather Station and Predictive AI Portal System focuses on real-time weather monitoring, data acquisition, machine learning-based forecasting, and emergency advisory dissemination. The system integrates IoT sensors, FastAPI-based backend services, SQLite database management, XGBoost prediction models, WebSocket communication, and interactive web dashboards. The primary objective is to provide hyperlocal weather information and short-term forecasting while ensuring efficient communication between monitoring stations and end users.

4.2 Existing System

Traditional weather forecasting systems generally rely on large-scale meteorological stations and numerical weather prediction models. These systems often suffer from the following limitations:

- Limited hyperlocal forecasting capability.
- High infrastructure and maintenance costs.
- Large computational requirements.
- Lack of real-time localized sensor integration.
- Limited short-term prediction and emergency broadcasting mechanisms.
- Reduced accessibility for rural and remote regions.

4.3 Proposed System

The proposed Intelligent Weather Station and Predictive AI Portal System is an IoT and AI-based platform designed to provide real-time weather monitoring and fifteen-minute advance forecasting.

The system consists of:

- DHT11 sensor for temperature and humidity measurement.
- DIY anemometer sensor for wind speed measurement.
- FastAPI backend server for data processing and communication.
- SQLite database for telemetry storage.
- XGBoost machine learning models for prediction.
- WebSocket communication for real-time updates.
- Base Station Dashboard for administration and monitoring.
- Client Portal for displaying weather information and alerts.
- Emergency advisory broadcasting system.

4.4 Feasibility Study

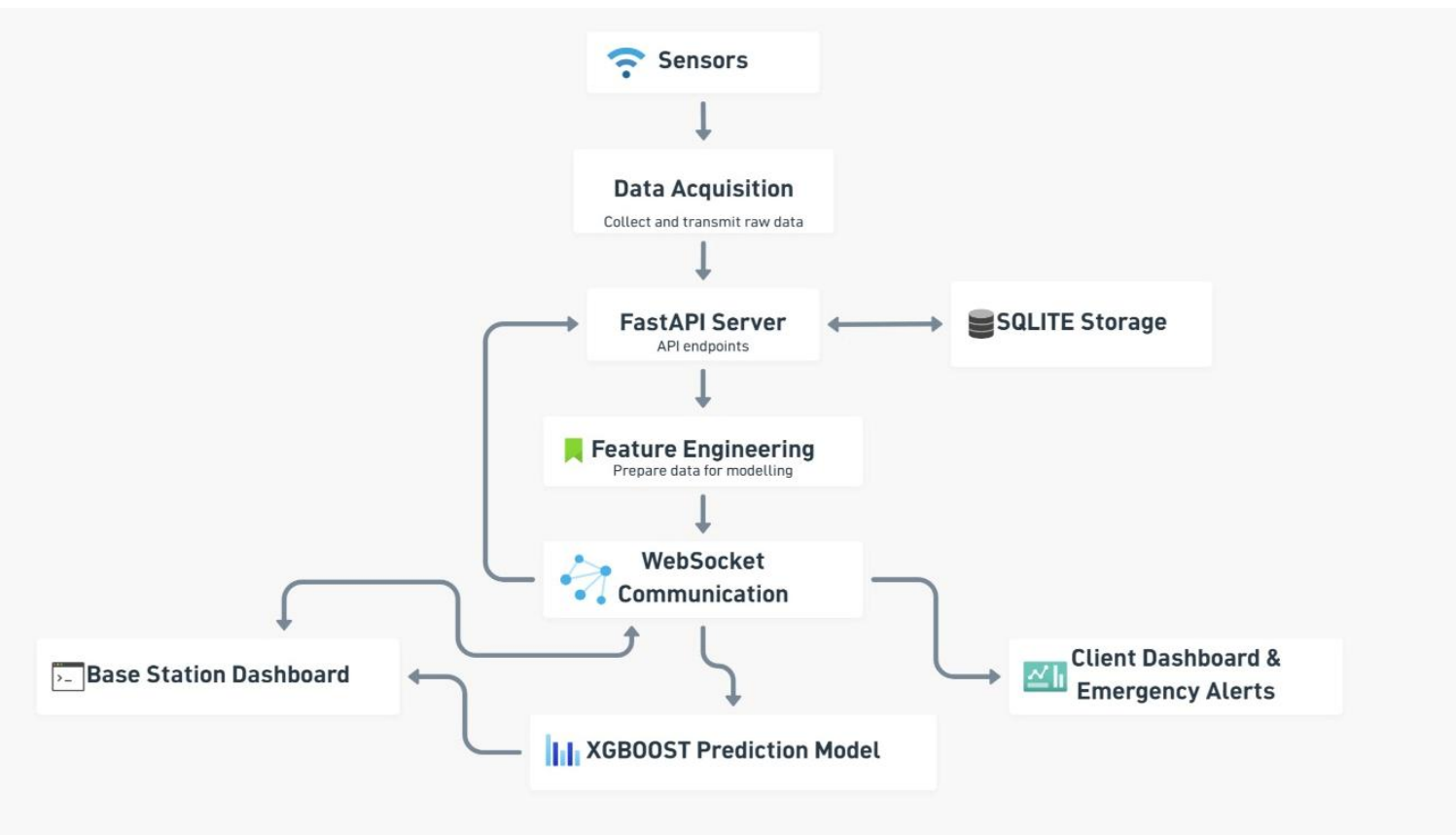
Technical Feasibility: The system utilizes open-source technologies and low-cost sensors, making implementation technically feasible.

Economic Feasibility: The use of DHT11, DIY anemometer, SQLite, Python, and open-source frameworks significantly reduces implementation costs.

Operational Feasibility: The system provides a user-friendly dashboard, real-time forecasting, and automated alert mechanisms, making it suitable for practical deployment

4.5 System Architecture

The proposed architecture follows a Client-Server model and consists of the following components:



○ Working Principle

Step 1: The DHT11 sensor measures temperature and humidity, while the anemometer measures wind speed.

Step 2: The sensor data is transmitted to the FastAPI server through WebSocket communication.

Step 3: The server stores the incoming telemetry in the SQLite database.

Step 4: Feature engineering techniques generate lag features, cyclic features, and statistical features from the collected data.

Step 5: The XGBoost models process the weather data and predict future temperature and humidity conditions fifteen minutes in advance.

Step 6: The predicted values and real-time sensor data are transmitted to the dashboards through WebSockets.

Step 7: The Base Station administrator can issue emergency weather advisories and broadcast alerts to all connected clients.

Step 8: The Client Portal displays real-time telemetry, weather forecasts, weather animations, and emergency notifications.

4.6 Database Design

The system utilizes SQLite as the local database management system.

Database Name: weather_station.db

Functions of Database:

- Storage of sensor telemetry data
- Storage of historical weather records
- Storage of prediction logs
- Storage of weather advisories and alerts.
- Storage of system event logs

4.7 Hardware Design

The hardware module is designed to collect environmental data and provide status indications. It consists of an Arduino Uno, a temperature sensor, an anemometer, two LED indicators, a protection diode, and a filter capacitor, as shown in Figure below.

The Arduino Uno acts as the main controller, receiving sensor data and processing it for transmission to the weather monitoring system. The temperature sensor measures ambient temperature, while the anemometer measures wind speed. The red and blue LEDs indicate system status and communication activity. A diode is used for reverse polarity protection, and a capacitor helps reduce electrical noise and stabilize sensor signals. The acquired sensor data is sent to the backend server for storage, machine learning-based forecasting, and real-time visualization on the client dashboards.

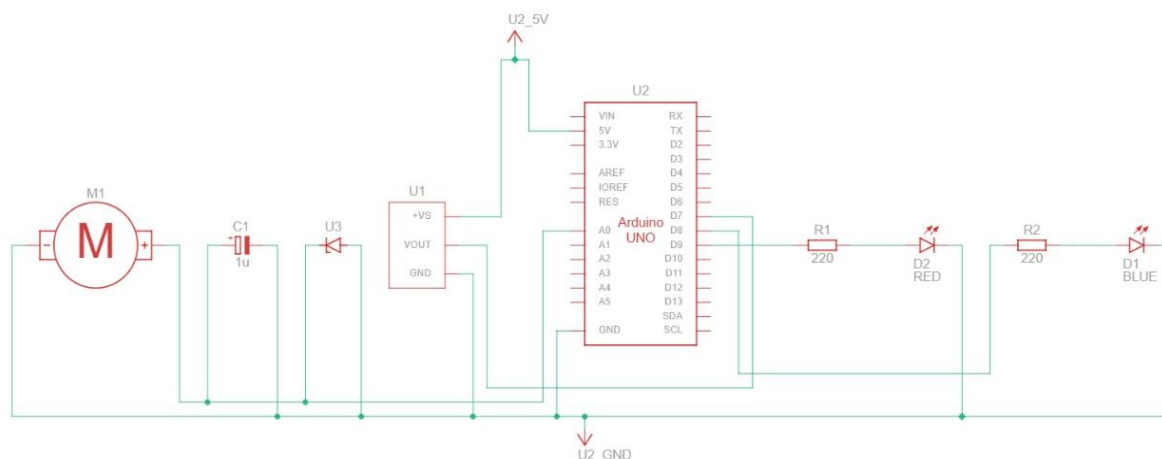


Fig: Circuit Connection Diagram

Component	Arduino Uno Connection
Temperature Sensor VCC	5V
Temperature Sensor GND	GND
Temperature Sensor OUT	Analog Pin A0
Anemometer Signal	Digital Pin D2
Anemometer VCC	5V
Anemometer GND	GND
Red LED (through 220 Ω resistor)	Digital Pin D7
Blue LED (through 220 Ω resistor)	Digital Pin D8
Capacitor	Across Anemometer VCC and GND
Diode	In series with the Anemometer signal line for protection

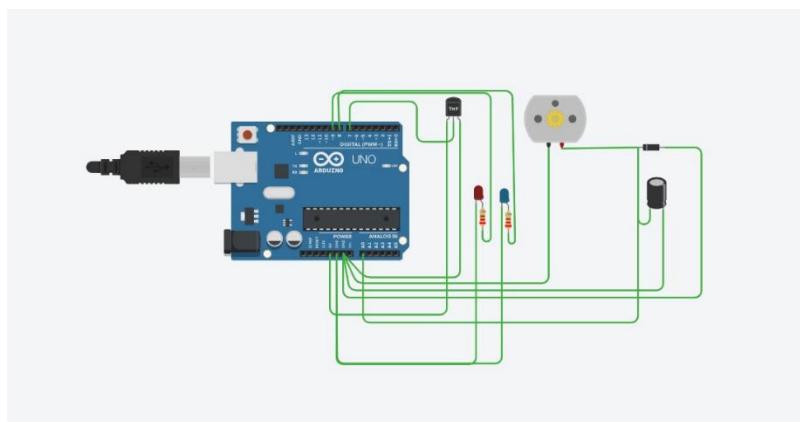


Fig: Hardware connection

4.8 Machine Learning Methodology

The forecasting engine utilizes XGBoost Regression models.

Prediction Models: *temp_model.pkl*, *humidity_model.pkl*

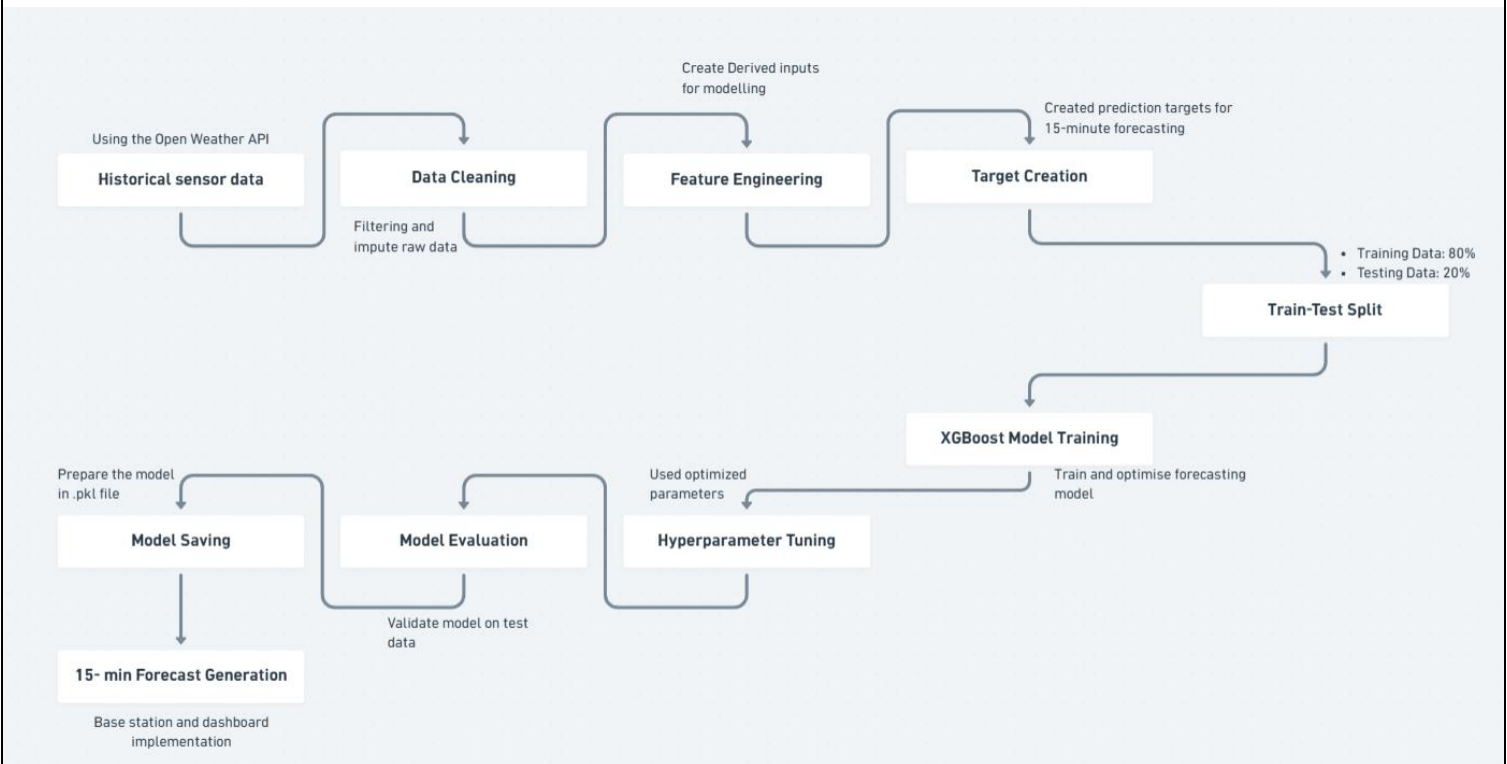
The models are trained using historical weather observations and engineered features.

Input Features:

- Current Temperature
- Current Humidity
- Wind Speed
- Time Features
- Lag Features
- Rolling Statistics

Predicted Outputs: Temperature and Humidity after 15 minutes

Machine Learning Workflow:



4.9 Backend Methodology

The backend server is developed using FastAPI and deployed using the Uvicorn ASGI server. It serves as the central processing unit of the proposed system by receiving sensor telemetry data, storing collected information in the SQLite database, performing feature engineering, executing machine learning models for weather prediction, generating fifteen-minute forecasts, managing

WebSocket-based real-time communication, broadcasting emergency advisories, and sending notifications to all connected clients.

4.10 WebSocket Communication Methodology

The system uses bidirectional WebSocket communication for real-time data transmission.

WebSocket Endpoints:

/ws/client

- Sends live telemetry data
- Sends prediction data
- Sends weather advisories
- Receives client acknowledgements

/ws/admin

- Sends administrative commands
- Receives manual overrides
- Broadcasts emergency alerts
- Updates system configurations

4.11 Frontend Implementation

The frontend dashboards are developed using Next.js and React and Styling- HTML5, CSS3, Custom CSS Variables

Base Station Dashboard- The Base Station Dashboard is developed using Next.js and provides administrative monitoring and control functionalities. It offers real-time telemetry visualization, weather forecast displays, access to system logs, an interface for broadcasting alerts and advisories, and a CSV export facility for downloading historical weather data and reports.

Client Portal- The Client Portal is designed to provide users with up-to-date weather information through an interactive interface. It displays real-time weather conditions, forecast reports, emergency notifications, weather visualizations, and advisory banners to keep users informed about current and predicted weather situations.

4.12 Emergency Advisory System

The system incorporates an Emergency Advisory System to disseminate critical weather information to connected clients in real time. Advisory messages are categorized into three severity levels: *Information*, *Warning*, and *Critical Danger*, allowing users to quickly assess the urgency of weather conditions. The system supports full-screen alert overlays, flashing warning interfaces, browser notifications, and real-time broadcasting to ensure that important alerts are

prominently displayed. Through WebSocket-based communication, emergency advisories are instantly synchronized across all connected client dashboards, enabling timely awareness and response to potentially hazardous weather events. Additionally, administrators can manually issue, update, or revoke advisories from the Base Station Dashboard, ensuring effective management of emergency communications.

4.13 System Workflow

- Environmental sensors collect weather parameters i.e. DHT11 and Anemometer sensor
- Sensor.py transmits telemetry to the FastAPI server using TCP Socket Port 5000.
- OpenWeatherMap API provides atmospheric pressure information.
- The backend stores telemetry data in the SQLite database.
- Feature engineering is performed on the collected data.
- Machine learning models predict future temperature and humidity values.
- The forecast engine generates weather predictions for the next fifteen minutes.
- Predictions and telemetry are transmitted through WebSocket communication.
- Client dashboards display real-time weather information and forecasts.
- Administrators may broadcast advisories and emergency alerts from the base station.
- Browser notifications and warning messages are instantly delivered to connected clients.

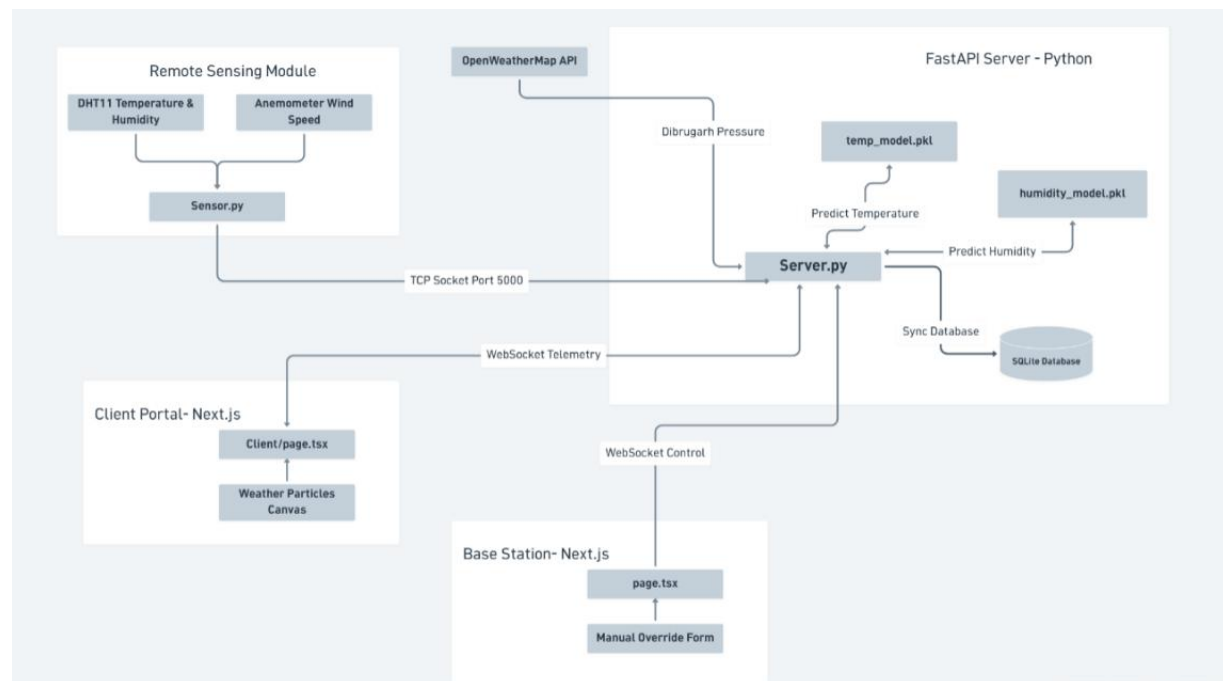


Fig: Complete System Flow

RESULTS, TESTING AND PERFORMANCE EVALUATION

This chapter presents the implementation results, system testing procedures, and performance evaluation of the Intelligent Weather Station and Predictive AI Portal System. The developed system was tested using real-time weather data collected from DHT11 and anemometer sensors along with historical weather data obtained from the OpenWeather API. The primary objective of testing was to evaluate the accuracy, reliability, and real-time performance of the forecasting and monitoring system.

5.1 System Testing

The system was tested under various environmental conditions to verify:

- Real-time sensor data acquisition.
- Database storage and retrieval.
- WebSocket communication.
- Machine learning prediction accuracy.
- Dashboard synchronization.
- Emergency advisory broadcasting.
- Weather visualization rendering.

5.2 Functional Testing

Test Case	Expected Result	Status
Sensor Data Acquisition	Temperature, Humidity, and Wind data collected successfully	Pass
Database Logging	Telemetry stored in SQLite database	Pass
WebSocket Communication	Real-time data synchronization	Pass
Temperature Prediction	Fifteen-minute prediction generated	Pass
Humidity Prediction	Fifteen-minute prediction generated	Pass
Advisory Broadcasting	Alerts displayed on client dashboard	Pass
Notification System	Browser notifications triggered	Pass
Dashboard Visualization	Weather animations rendered correctly	Pass

5.3 Sample Output of System

Current Weather Conditions

Temperature : 28.5 °C

Humidity : 78 %
 Wind Speed : 1.4 m/s
 Forecast After 15 Minutes
 Predicted Temperature : 28.8 °C
 Predicted Humidity : 80 %
 Weather Trend : Cooling Trend / Rain Expected

5.4 Performance Evaluation Metrics

The performance of the machine learning models was evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Coefficient of Determination (R^2 Score)

Parameter	MAE	RMSE	R^2 Score	Performance	Intepretation
Temperature	1.075	1.292	0.751	Good	The model explains approximately 75.1% of the variation in temperature and demonstrates good short-term forecasting capability.
Humidity	3.724	4.972	0.781	Good	The model explains approximately 78.1% of the variation in humidity and provides reliable short-term humidity forecasting.
Pressure	0.568	0.782	0.413	Moderate	The model demonstrates moderate predictive capability for atmospheric pressure forecasting.
Wind Speed	0.553	0.840	0.450	Moderate	The model provides moderate prediction performance for wind speed forecasting.
Cloud Cover	5.538	8.203	0.712	Good	The model successfully captures cloud cover variations and achieves good forecasting performance.

Rainfall	0.043	0.122	- 0.00018	Poor	Rainfall prediction performance is limited due to the highly imbalanced and insufficient local rainfall dataset.
----------	-------	-------	--------------	------	--

5.5 System Achievements

- Real-time weather monitoring using IoT sensors.
- Fifteen-minute advance weather forecasting using Machine Learning.
- Successful implementation of WebSocket-based telemetry.
- Real-time dashboard synchronization.
- Emergency advisory broadcasting system.
- Interactive weather visualization using Canvas animations.
- Hyperlocal forecasting using localized weather datasets.
- Low-cost and scalable architecture suitable for practical deployment.

5.6 Discussion

The developed Intelligent Weather Station and Predictive AI Portal System successfully demonstrates the integration of IoT, Artificial Intelligence, and real-time web technologies for hyperlocal weather monitoring and forecasting. The experimental results indicate good prediction capability for temperature, humidity, and cloud cover, while pressure and wind forecasting show moderate performance. Rainfall prediction remains challenging due to limited and imbalanced local datasets.

The system successfully generated fifteen-minute advance weather forecasts and provided real-time monitoring, emergency advisories, and interactive visualization, making it suitable for applications such as smart agriculture, environmental monitoring, and disaster management systems.

CONCLUSION AND FUTURE SCOPE

The Intelligent Weather Station and Predictive AI Portal System successfully demonstrates the integration of Internet of Things (IoT), Artificial Intelligence (AI), and Machine Learning (ML) technologies for real-time weather monitoring and short-term forecasting. The system utilizes DHT11 and anemometer sensors to collect environmental data and employs XGBoost machine learning models to generate fifteen-minute advance predictions of temperature and humidity.

The implementation of FastAPI, WebSocket communication, and SQLite databases enables efficient real-time telemetry processing, data storage, and synchronization between the Base Station and Client Portal. The developed dashboards provide live weather monitoring, predictive analytics, emergency advisory broadcasting, and interactive weather visualizations.

Experimental results indicate good forecasting performance for temperature, humidity, and cloud cover predictions while maintaining moderate performance for pressure and wind speed forecasting. The system offers a low-cost, scalable, and practical solution for hyperlocal weather forecasting and can be effectively utilized in smart agriculture, environmental monitoring, disaster management, and educational applications.

7.1 Gap Analysis

The developed Intelligent Weather Station and Predictive AI Portal System demonstrates satisfactory performance for temperature, humidity, and cloud cover forecasting. However, several gaps exist between the current system capabilities and the requirements of a highly accurate, scalable, and production-ready weather forecasting platform.

Area	Current Status	Identified Gap	Impact
Temperature Prediction	Good performance ($R^2 = 0.751$)	Accuracy can still be improved using larger datasets and advanced models	Moderate forecasting errors
Humidity Prediction	Good performance ($R^2 = 0.781$)	Limited by sensor precision and environmental noise	Reduced reliability in varying conditions
Pressure Prediction	Moderate performance ($R^2 = 0.413$)	Current model fails to capture complex pressure variations	Lower forecast accuracy
Wind Speed Prediction	Moderate performance ($R^2 = 0.450$)	Limited sensor sensitivity and inadequate training data	Inaccurate wind forecasting
Cloud Cover Prediction	Good performance ($R^2 = 0.712$)	Absence of satellite imagery and visual weather features	Limited prediction capability

Rainfall Prediction	Poor performance ($R^2 \approx 0$)	Highly imbalanced and insufficient rainfall dataset	Very low prediction accuracy
Sensor Data Acquisition	Basic environmental sensors used	Low sensor accuracy and calibration limitations	Noisy and inaccurate input data
Hardware Integration	IoT hardware connected to cloud platform	Incomplete synchronization between hardware and software modules	Data loss, delays, and reliability issues
AI Model Training	Trained on limited local datasets	Insufficient historical weather data for robust learning	Reduced generalization ability
Forecasting Horizon	Limited to short-term (15-minute) forecasting	Lack of long-term prediction capability	Restricted practical usability
Geographic Coverage	Single-location deployment	Cannot generalize to multiple climatic regions	Limited scalability
Real-Time Reliability	Depends on internet connectivity	Vulnerable to network interruptions	Reduced system availability

Discussion of Identified Gaps

- Limited Rainfall Prediction Accuracy

The rainfall forecasting model exhibits poor performance with an R^2 score close to zero. This is primarily due to the highly imbalanced nature of rainfall events and the lack of sufficient local rainfall records. The model struggles to learn meaningful rainfall patterns, resulting in unreliable predictions.

- Low Sensor Accuracy and Data Quality

The quality of machine learning predictions is directly dependent on the quality of sensor data. Low-cost sensors may introduce measurement errors, drift, calibration issues, and environmental noise. These inaccuracies propagate through the forecasting model and reduce overall prediction performance.

- Inadequate Hardware–Software Integration

Although IoT sensors, cloud infrastructure, and AI modules are integrated, communication delays, synchronization issues, and occasional data transmission failures create a gap between data collection and prediction generation. A more robust middleware architecture is required for seamless operation.

- Insufficient Training Dataset

The AI models are trained using a relatively small local weather dataset. Limited historical records reduce the model's ability to learn seasonal, extreme-weather, and rare-event patterns. This directly affects the accuracy of pressure, wind speed, and rainfall forecasting.

- Moderate Performance for Pressure and Wind Speed

The pressure and wind speed models achieve only moderate prediction performance. These atmospheric parameters are influenced by complex meteorological interactions that are not fully captured by the current feature set and machine learning approach.

- Lack of Advanced Weather Data Sources

The present system relies mainly on ground-based sensor measurements. The absence of satellite imagery, radar information, and regional meteorological datasets restricts forecasting accuracy and limits environmental awareness.

- Limited Forecasting Capability

The current implementation focuses on short-term forecasting. Long-term hourly and daily weather predictions require larger datasets, more sophisticated models such as LSTM and GRU, and greater computational resources.

7.2 Applications

- Hyperlocal weather monitoring and forecasting.
- Smart agriculture and irrigation management.
- Disaster management and early warning systems.
- Environmental and climate monitoring.
- Smart city weather information systems.
- Educational and research applications.
- Renewable energy and transportation planning.
- Real-time weather advisory and alert systems.

7.3 Limitations

- Prediction accuracy decreases during sudden and extreme weather changes.
- Rainfall prediction is limited due to insufficient and imbalanced local datasets.
- The system depends on stable internet connectivity and continuous sensor operation.
- Sensor accuracy and environmental noise may affect forecasting performance.
- The current system is optimized primarily for short-term (15-minute) forecasting.

7.4 Future Scope

- Extend forecasting from 15-minute predictions to hourly and daily forecasts.
- Integrate additional sensors and satellite weather data for improved accuracy.
- Implement advanced deep learning models such as Long Short-Term Memory and Gated Recurrent Unit.
- Develop mobile applications for real-time weather monitoring and alerts.
- Expand the system to support multiple geographical locations.
- Incorporate severe weather warning systems for storms, floods, and heat waves.
- Deploy the system for smart agriculture and large-scale environmental monitoring applications.

7.5 Overall Contribution

The proposed system contributes to the development of an intelligent, low-cost, and scalable hyperlocal weather forecasting platform by integrating IoT sensors, real-time communication technologies, machine learning models, and interactive dashboards into a unified system capable of generating reliable short-term weather forecasts and emergency advisories.

REFERENCES:

Research

- [1] Gebretinsae et al., Machine Learning-Based Intelligent Weather Monitoring and Predicting System, 2022.
- [2] Singh et al., IoT Based Machine Learning Weather Monitoring and Prediction Using WSN, 2024.
- [3] Bhavani et al., Smart Weather Monitoring and Prediction System Using IoT and AI, 2025.
- [4] Kukre et al., Integrated IoT Based Weather Monitoring and Machine Learning Weather Prediction System, 2025.
- [5] Isler et al., Fog-Enabled Machine Learning Approaches for Weather Prediction in IoT Systems, 2025.
- [6] Das and Nayak, Integration of IoT-AI Powered Local Weather Forecasting: A Game-Changer for Agriculture, 2025.

Tools & Documentation:

OpenWeather API Documentation (<https://openweathermap.org/api>)
Python Official Documentation (<https://www.python.org/doc/>)
XGBoost Documentation (<https://xgboost.readthedocs.io/en/stable/>)
FastAPI Documentation (<https://fastapi.tiangolo.com/>)
Arduino Documentation (<https://docs.arduino.cc/>)
Arduino IDE (Version 2.3.6) (<https://www.arduino.cc/en/software>)
Next.js Documentation (<https://nextjs.org/docs>)
SQLite Documentation (<https://www.sqlite.org/docs.html>)
Antigravity IDE Official Website (<https://antigravity.tech/in>)

APPENDIX

-Appendix A

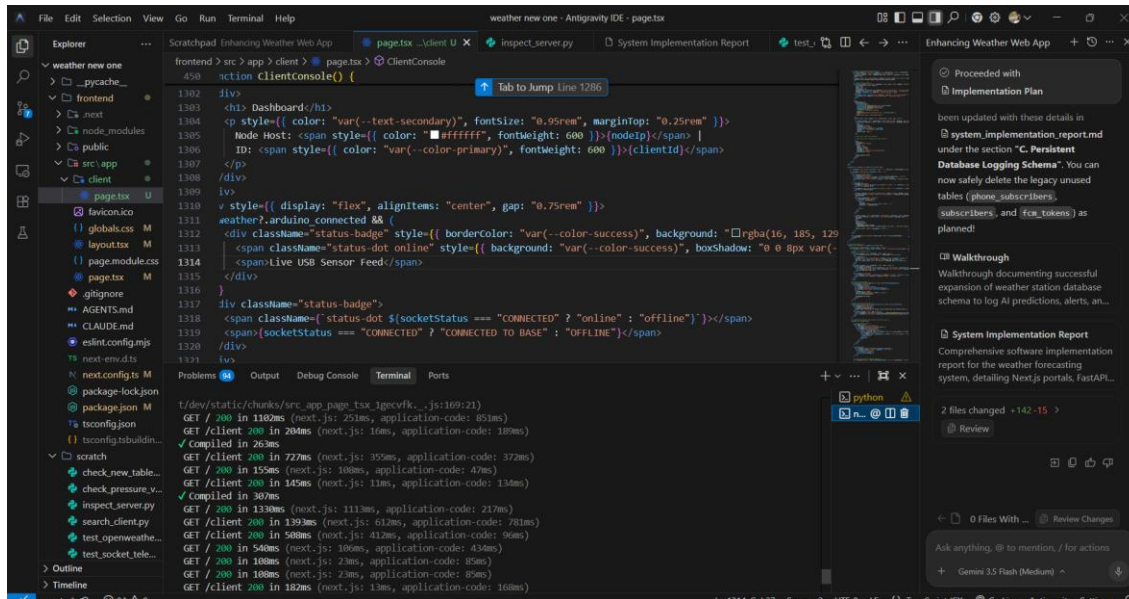


Image: Antigravity Interface

This figure illustrates the Antigravity Interface used to develop, execute, and monitor the proposed weather forecasting system, integrating both the FastAPI backend and Next.js frontend within a unified environment.

-Appendix B

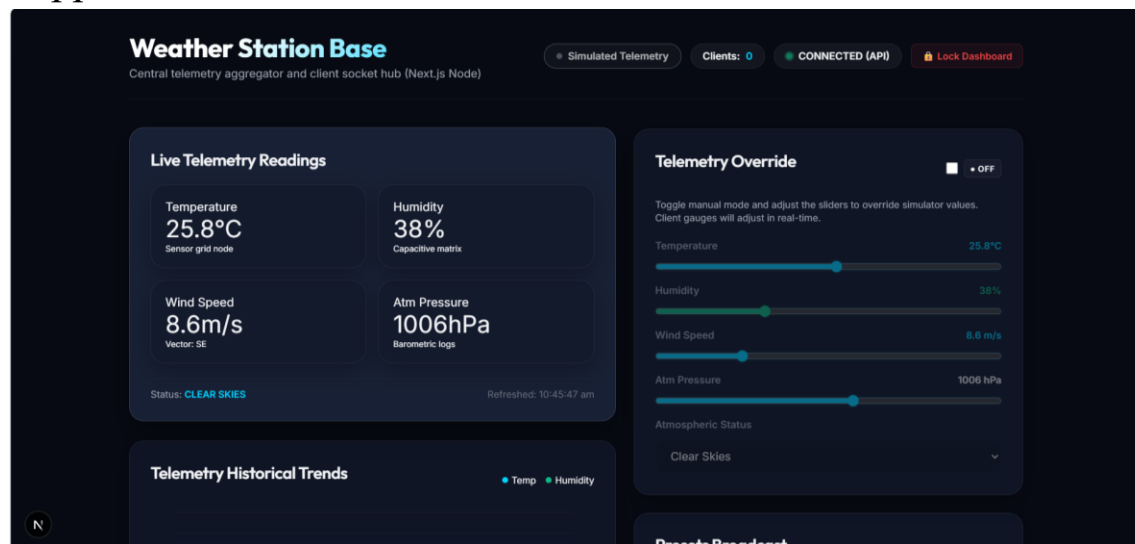


Image: Base Station Monitoring Dashboard

This figure illustrates the Base Station Monitoring Dashboard, which enables administrators to monitor real-time telemetry, visualize forecasts, manage alerts, review system logs, and export weather data.

-Appendix C

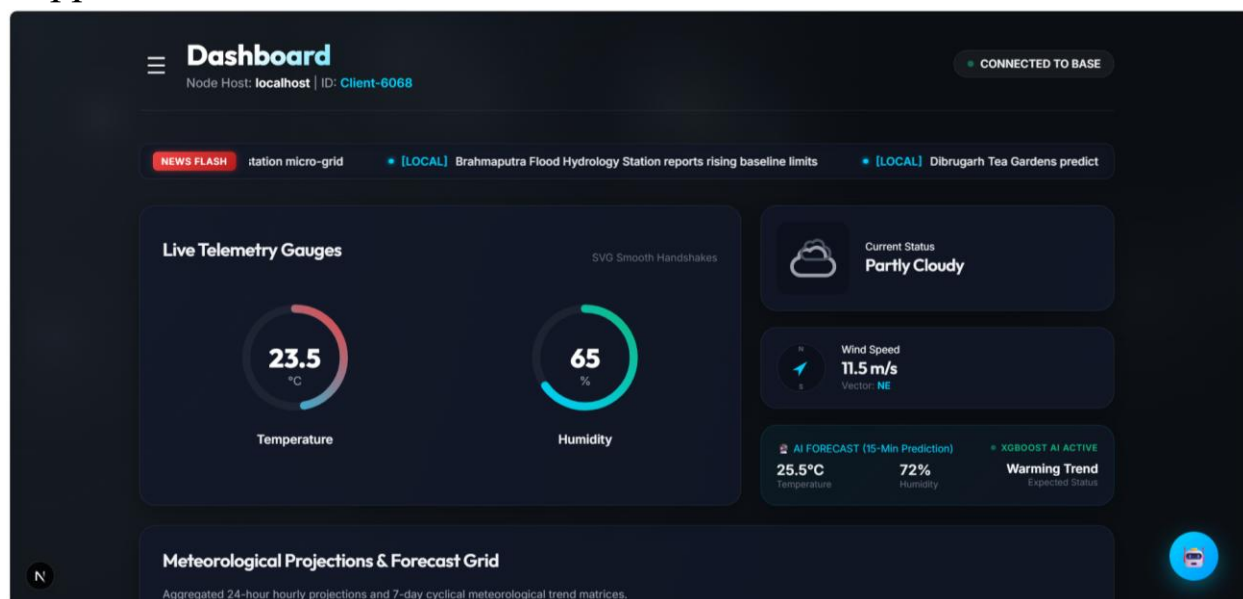


Image: Client Monitoring Dashboard

This figure illustrates the Client Monitoring Dashboard, which provides users with real-time weather information, forecast reports, weather visualizations, emergency notifications, and advisory updates.

-Appendix D

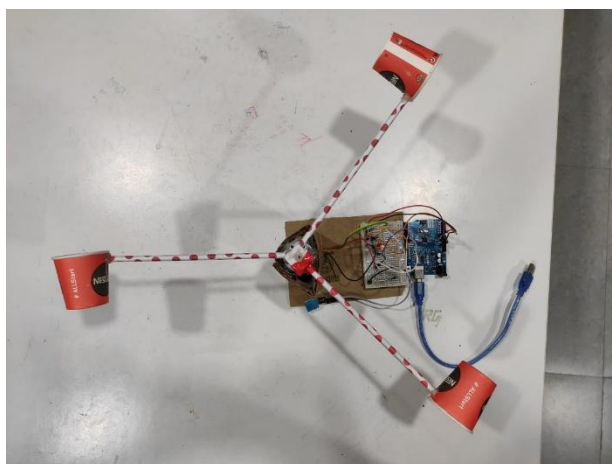
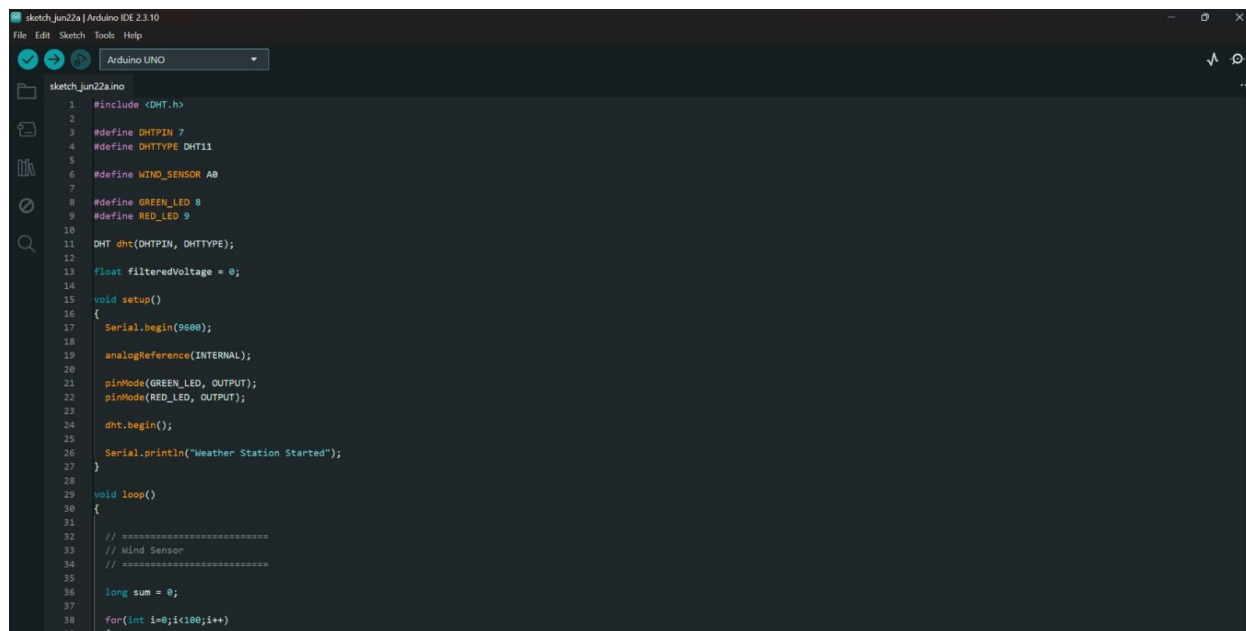


Fig: Monitoring Hardware circuit

This figure illustrates the hardware circuit of the proposed weather monitoring system, comprising the Arduino Uno, temperature sensor, anemometer, indicator LEDs, protection diode, and filtering capacitor used for environmental data acquisition and system status indication.

-Appendix E



```

sketch_jun22a.ino
1  #include <DHT.h>
2
3  #define DHTPIN 7
4  #define DHTTYPE DHT11
5
6  #define WIND_SENSOR A0
7
8  #define GREEN_LED 8
9  #define RED_LED 9
10
11 DHT dht(DHTPIN, DHTTYPE);
12
13 float filteredVoltage = 0;
14
15 void setup()
16 {
17   Serial.begin(9600);
18
19   analogReference(INTERNAL);
20
21   pinMode(GREEN_LED, OUTPUT);
22   pinMode(RED_LED, OUTPUT);
23
24   dht.begin();
25
26   Serial.println("Weather Station Started");
27 }
28
29 void loop()
30 {
31
32   // *****
33   // Wind Sensor
34   // *****
35
36   long sum = 0;
37
38   for(int i=0;i<100;i++)
39   {

```

Fig: Arduino Program for Hardware remote sensing connection

This figure illustrates the Arduino program developed for the remote sensing module, which acquires environmental data from connected sensors, processes the measurements, and transmits telemetry information to the backend weather forecasting system for real-time monitoring and prediction.